typesense

Recommendations

Copy Markdown

Typesense can generate recommendations based on the actions users take in a given session, using the Vector Search feature.

This involves building a Machine Learning model to generate embeddings, storing them in Typesense, and then doing a nearest-neighbor search in Typesense.

In this article's we'll talk about how to use the Starspace ML model to generate embeddings.

Transformers4Rec is another ML model that can be used for this use-case, among others.

Scenario

We'll use an e-commerce products dataset in this article for illustration, but the concepts below can be applied to any domain (eg: recommending articles, movies, or any type of records stored in Typesense).

Step 1: Prepare training dataset

We'll be using Starspace to build our ML model.

Starspace expects the training dataset in the following format - one line for each user session and the set of items that they interacted with in that session.

session-data.txt

```
apple orange banana broccoli mango
cereals soda bread nuts cookies
tissue detergent butter cheese milk eggs
ice_cream milk pancake_mix muffins
```

In the example above, the first line indicates that a certain user interacted with (viewed, bought, added to cart, etc) the products `apple`, `orange`, `banana`, `broccoli` and `mango` in a single

`soda`, `bread`, `nuts` and `cookies` in another session.

> TIP
>
> We're using the `product_name` in this example to make this article easier to read. In a production setting, you'd want to use the product's ID or SKU in the training dataset.

Step 2: Setup Starspace

Install system dependencies

Ensure that you have a C++11 compiler (gcc-4.6.3 or newer or Visual Studio 2015 or clang-3.3 or newer).

On macOS you'd need to install XCode and on Linux distros, you'd need to install the `build-essential` package from your distro's package manager.

Clone Starspace source code

```
git clone https://github.com/facebookresearch/Starspace.git
cd Starspace
```

Setup Boost

Boost is a library required by Starspace.

From inside the `Starspace` directory above, run the following:

```
curl -LO https://boostorg.jfrog.io/artifactory/main/release/1.82.0/source/boost_1_82_
tar -xzvf boost_1_82_0.tar.gz
```

Compile Starspace

```
make -e BOOST_DIR=boost_1_82_0 && \
  make embed_doc -e BOOST_DIR=boost_1_82_0
```

To verify that Starspace is working fine, if you run `./starspace`, you should see output similar to the following:

```
$ ./starspace
Usage: need to specify whether it is train or test.

"starspace train ..."  or "starspace test ..."
...
```

Step 3: Train Starspace model

Name the file with your training dataset from Step 1 as `session-data.txt`.

Then run the following command to train your model:

```
./starspace train \
  -trainFile <path/to/session-data.txt> \
  -model productsModel \
  -label '' \
  -trainMode 1 \
  -epoch 25 \
  -dim 100
```

Once this command runs, it will generate two files - a binary file called `productsModel` and a TSV file with model weights called `productsModel.tsv`.

You want to run this training step periodically as you collect new session data as users use your site / app.

Step 4: Generate embeddings

First extract all the unique products from our training dataset:

For each product, generate embeddings and store them in a JSONL file:

```sh
echo -n > ${output_jsonl_file}

while read -r item; do
    embedding=$(echo "${item}" | ./embed_doc productsModel | tail -1 | tr ' ' ',')
    echo "{\"id\":\"${item}\",\"embedding\":[${embedding%?}]}" >> "${output_jsonl_fi
done <<< "${unique_items}"
```

This will generate a JSONL file that looks like this:

products-with-embeddings.jsonl

```
{"id":"apple","embedding":[-0.129717,0.173566,0.105385,0.0413297,-0.0290213,-0.0255852,0.08
{"id":"banana","embedding":[-0.151976,0.167556,0.0984403,0.0582992,-0.0267645,-0.0632901,0.
...
```

We can now ingest this JSONL file into Typesense.

Step 5: Index the embeddings in Typesense

```sh
export TYPESENSE_API_KEY=xyz
export TYPESENSE_URL='https://xyz.a1.typesense.net'
```

Create a collection:

```sh
curl "${TYPESENSE_URL}/collections" \
       -X POST \
       -H "Content-Type: application/json" \
       -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
       -d '{
         "name": "products",
         "fields": [
           {"name": "name", "type": "string", "optional": true},
```

```
        {"name": "embedding", "type": "float[]", "num_dim": 100 }
      ]
    }'
```

Notice how we've set `num_dim: 100`. This correlates to the `-dim 100` parameter we set when training our Starspace model.

Import the JSONL file with embeddings into the collection:

```sh
curl -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
      -X POST \
      -T products-with-embeddings.jsonl \
      "${TYPESENSE_URL}/collections/products/documents/import?action=emplace"
```

We're only inserting the embeddings for each product here. You can also import other values like name, description, price, categories, etc separately using the same `id` field as reference to populate the rest of the product record.

Step 6: Generate recommendations

Now let's say a user lands on our site / app and interacts with the following products in their session:

```
mango broccoli milk
```

To generate recommendations based on this session data, let's first generate embeddings:

```
export embedding=$(echo "mango broccoli milk" | ./embed_doc productsModel | tail -1
```

```
echo ${embedding}
-0.0862846,0.127956,0.0558543,0.0745331,0.02449,-0.131018,0.0886827,-0.0571893,-0.03
```

recommend to this user:

```
curl "${TYPESENSE_URL}/multi_search" \
        -X POST \
        -H "Content-Type: application/json" \
        -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
        -d '{
          "searches": [
            {
              "q": "*",
              "collection": "products",
              "vector_query": "embedding:(['${embedding%?}'], k:10, distance_thresho
            }
          ]
        }' | jq '.results[0].hits[] | .document.id'
```

This will return the following recommendations, which we can show this user in our UI, after filtering out items they've already seen in this session:

```
"broccoli"
"mango"
"banana"
"apple"
"orange"
"tissue"
"detergent"
"cheese"
"milk"
"butter"
```

> **TIP**
>
> We're using the product's name in this example to make this article easier to read. In a production setting, you'd want to use the product's ID or SKU in the training dataset and to generate embeddings like this:
>
> ```
> sku_1 sku_4 sku_5
> sku_5 sku_8 sku_1 sku_2 sku_10
> ```

typesense

⌘ K

Edit page

Last Updated: 8/31/2023, 11:03:18 AM

← Semantic Search

Personalization →